Ministry of Higher Education
and Scientific Research

EPI
Private International Higher
School of Polytechnic

ACADEMIC YEAR
2024/2025

END OF STUDIES PROJECT

In order to obtain the National Diploma in Engineering

Field of study:

Entitled

Internship place **Author** **Supervisor**

Dedication

This project has been made possible thanks to the support and motivation from many individuals to whom we would like to express our deepest gratitude.

We extend our thanks to our beloved parents, who have always been there for us with their support and encouragement.

We also want to express our appreciation to our friends, whose constant presence and motivation have been crucial through this journey.

Thank you all for your support.

Acknowledgments

We extend our sincere thanks to all the professors and individuals who guided us and provided their support during our research.

First, we express our deep gratitude to our supervisor, Mrs. Rania MKHININI, for her patience and wise counsel, which have greatly boosted our work.

We also thank the jury members for taking the time to discuss our project and for their insightful comments.

Special thanks to Mr. Tarek HAMDANI for honoring us by chairing the jury. We hold him in high regard and respect.

We are also grateful to Mrs. Wiem BELHEDI for agreeing to be a part of the jury and evaluate our work.

Thank you all for your support and contributions.

Table of Contents

List of Figures

List of Tables

List of abbreviations

API	Application Programming Interface
POD	Print-On-Demand
SPA	Single-Page Application
UML	Unified Modeling Language
RBAC	Role-Based Access Control
UI	User Interface
JWT	JSON Web Token
REST	Representational State Transfer
ORM	Object-Relational Mapping
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
JSON	JavaScript Object Notation
CSV	Comma-Separated Values

General Introduction

In contemporary digital commerce, platforms increasingly rely on flexible fulfillment models that reduce inventory risk while preserving product variety and personalization. Print-On-Demand (POD) addresses these needs by producing items only after an order is placed, enabling a marketplace to offer a broad catalog without stock management. However, building a POD marketplace is not a purely e-commerce problem: it requires coordinating multiple roles with distinct incentives, enforcing role-based access, ensuring secure transactions, and supporting an order lifecycle that connects design publishing, product customization, production, shipping, and customer feedback.

Within this context, this report presents *printymand*, a full-stack POD marketplace platform inspired by creator marketplaces. The platform connects customers, designers, printers, and administrators in a unified ecosystem. Customers can browse and customize products, select a printer, and track orders. Designers can upload and publish artworks and earn royalties. Printers can subscribe to plans that affect their visibility and capacity, manage incoming orders, and update fulfillment statuses. Administrators can moderate content and monitor system-wide analytics. The platform is implemented with an Angular single-page application, a NestJS RESTful backend API, and a PostgreSQL database, using JWT authentication and role-based authorization.

To better organize the work, this report is structured as follows.

- The first chapter "**Preliminary Study**" introduces the project and its motivation. It outlines objectives, presents a comparative view of representative marketplace approaches, and positions the proposed solution according to the specification.
- The second chapter "**Architecture and Planning**" details the requirements analysis, the architectural choices, and the planning approach used to organize development.
- The third chapter "**Spiral Cycle 1**" presents the first development cycle, focusing on foundational marketplace capabilities such as authentication, role onboarding, catalog discovery, and the baseline design-asset pipeline.
- The fourth chapter "**Spiral Cycle 2**" discusses the second development cycle, emphasizing product customization, printer selection, checkout/payment confirmation, and order tracking.
- The fifth chapter "**Spiral Cycle 3**" covers consolidation-oriented work including governance (administration and moderation), exports/invoices, and platform optimization.

Finally, the report concludes with a general conclusion summarizing achieved objectives, technical contributions, and perspectives for future enhancement.

Chapter 1

Preliminary study

Introduction

This chapter initiates our project study, where we will outline the project, its domain, scope, and importance. We begin with a thorough analysis of the existing landscape to identify the challenges that our Print-on-Demand (POD) marketplace seeks to address, the objectives it aims to achieve, and the methodology we have selected for its implementation.

1.1 General overview

The project "Printymand" is carried out as part of the Integration Project for the academic year 2025-2026. Printymand is a Tunisian web-based print-on-demand (POD) marketplace platform: POD allows products to be printed and fulfilled on demand from digital designs, removing the need for inventory. This web platform will connect designers, local printers and suppliers to offer locally produced, culturally relevant goods, provide designer storefronts and royalties, and support localized payments with a bilingual web interface.

1.2 Problem

Although print-on-demand removes inventory requirements, Tunisian creators and buyers face concrete barriers that impede adoption and growth. International marketplaces frequently exclude Tunisia or impose high shipping costs and long delivery times, while payment and payout systems commonly used by these platforms are often unavailable or impractical for local users. Local providers, on the other hand, generally lack dedicated designer-facing features (storefronts, royalty mechanisms), offer a narrow product catalog and limited seller tools, and operate as in-house printers with constrained fulfilment capacity — factors that prevent them from scaling to larger volumes or supporting broader regional demand. Together, these issues limit market access, designer monetization, and the overall scalability of POD in Tunisia.

1.3 Existing Solutions Study, Criticism, and Proposed Solution

1.3.1 Overview of Existing Solutions

The following short profiles present representative local and international POD providers referenced in this study.

1.3.1.1 MarYouli

MarYouli is a Tunisian service focused on personalized clothing (for example, t-shirts and hoodies). It primarily serves local customers who require simple customization and direct purchase.

1.3.1.2 TeeWinek

TeeWinek offers personalized gifts and apparel within Tunisia, with an emphasis on straightforward customization and social-media-driven sales.

1.3.1.3 TeePublic

TeePublic is an international creator marketplace that enables designers to open storefronts and sell designs across a broad product catalog; fulfilment and shipping are handled by third-party partners.

1.3.1.4 Redbubble

Redbubble is a global marketplace for independent artists, providing an extensive product range and automated order fulfilment through partner networks.

1.3.2 Critical Analysis of Existing Solutions

1.3.2.1 MarYouli

Strengths:

- Local production and delivery enable quick shipping within Tunisia.
- Offers decent print quality and customization for personal gifts.
- Operates in the local currency, simplifying payments for Tunisian customers.

Weaknesses:

- No designer marketplace: Designers cannot open a storefront or earn royalties.
- Limited product catalog: Mainly t-shirts and apparel, lacking accessories or lifestyle items.

- Outdated UI/UX: The interface feels basic, with limited interactivity and poor appeal to younger users.
- Manual operations: Order tracking and updates lack automation.
- No analytics or marketing tools: Designers or business users cannot monitor performance or visibility.
- They are printers that handle printing themselves, whereas our platform will generate profit for printers, designers, and apparel blanks suppliers.

1.3.2.2 TeeWinek

Strengths:

- Localized pricing and delivery; easily accessible for Tunisian customers.
- Simple customization process for personal designs or slogans.
- Active on social media, building some brand recognition.

Weaknesses:

- No designer collaboration model: The site doesn't allow creators to post and sell their artwork through personal storefronts.
- Poor UI/UX: The design interface and website layout are not engaging; navigation is confusing and lacks a professional e-commerce look.
- Narrow product focus: Limited mainly to apparel and basic items like mugs or phone cases.
- Lack of automation: Orders and production appear to rely on manual processes.
- Fulfillment capacity constraint: operates as an in-house/local printer with limited production, limiting the ability to handle large order volumes.
- They are printers that handle printing themselves, whereas our platform will generate profit for printers, designers, and apparel blanks suppliers.

1.3.2.3 TeePublic

Strengths:

- Full creator marketplace: Designers can open storefronts, upload artwork, and automatically receive royalties per sale.
- Large product range: Hundreds of items from clothing to accessories and home décor.
- Integrated marketing: Products can be promoted through social media or embedded in other stores.
- Automated fulfillment: Orders, printing, and shipping are handled seamlessly.

Weaknesses:

- No shipping to Tunisia: Tunisian users often cannot place orders or face very high shipping costs.
- Payment barriers: Requires PayPal or other platforms not fully supported in Tunisia.
- Cultural mismatch: Artwork and promotional content primarily target Western customers.
- No local pricing: Prices are based on foreign currencies, making it inaccessible to Tunisian buyers.

1.3.2.4 Redbubble

Strengths:

- Massive global reach: Millions of customers and active artists worldwide.
- Extensive catalog: Over 70 product types, from clothing to wall art and accessories.
- Automated workflow: Fully managed printing, shipping, and customer service.
- Strong brand recognition: Trusted by both artists and buyers internationally.

Weaknesses:

- Tunisia shipping issues: Redbubble either does not ship to Tunisia or has very slow and expensive delivery.
- Payment incompatibility: Payouts via PayPal or bank transfer are inaccessible for most Tunisian creators.
- High competition: Millions of designs make it difficult for new artists to stand out.
- Cultural disconnect: Western-centric aesthetics and pricing limit relevance to Tunisian consumers.

1.3.3 Identified Gap

From the comparative analysis, the following gaps are clear:

- There is no local platform in Tunisia that offers both local production & shipping and an open designer-marketplace/ store-front + royalty payments.
- Existing local platforms have weaknesses in UI/UX, which reduces appeal for creatives/designers.
- International marketplace platforms are often inaccessible (shipping/payout issues) to Tunisian designers and customers.
- Product diversification is limited in local solutions; designers lack tools, dashboards, and branding control.
- There is no platform in Tunisia that integrates printers, suppliers, and designers into a collaborative ecosystem; existing local solutions are standalone printers without such

partnerships.

1.3.4 Proposed Solution: Printymand

After studying the POD market and the existing solutions, Printymand will address the identified gaps by creating a Tunisian web-based POD marketplace platform focused on local shipping and local designers, with the following key features and design choices:

1.3.4.1 Key Features

Designer Storefronts and Royalties

Designers can register, open a personal storefront, upload their own designs, set prices, and receive royalties on each sale.

Local Production and Shipping Only

All products are produced and shipped locally within Tunisia. No international fulfilment; complying with local laws and reducing logistical complexity.

Diverse Product Catalog

Printymand will offer a comprehensive range of popular items: t-shirts, mugs, stickers, hoodies, tank tops, hats, bags, and similar apparel or accessories, a complete but focused catalog for wearable and collectible items.

Localized Payments and Payouts

Use payment methods acceptable in Tunisia; simplify payout to designers via bank transfer or other feasible local channels.

High Quality UI/UX

Modern, attractive website/app design that aligns with the aesthetic expectations of creatives and younger users. Smooth workflow for design upload, mockups, storefront management, browsing and buying.

Dual Language Interface

Offer interface in Arabic / French and optionally English, to serve local Tunisian users effectively.

Collaborative Ecosystem

Printymand integrates printers, apparel blanks suppliers, and designers into a unified platform, generating profit for all stakeholders through partnerships, marketplace fees, and shared revenue models.

1.3.4.2 Strategic Advantage

By focusing solely on the Tunisian market, Printymand can offer faster delivery, lower shipping cost, simplified payments, and better cultural relevance.

Allowing designers to earn regularly via storefronts will attract creative talent that currently lacks local platforms to monetize their designs.

A better UI/UX will help in distinguishing Printymand from local competitors whose sites are functional but not polished.

1.4 The adopted methodology

Choosing the right methodology is crucial for the success of any project. It provides an efficient and structured way of achieving objectives and allows the management of deadlines and the quality control of output.

1.4.1 The choice of the Spiral Model

The Spiral Model is a risk-driven, iterative software development methodology that combines elements of both design and prototyping in stages. It is particularly suitable for large and complex projects where requirements may evolve over time, and certain technical or business risks must be analyzed early. The development process progresses in successive cycles—called spirals—each consisting of four key activities: defining objectives, identifying and resolving risks, developing and validating components, and planning the next iteration. This allows for continuous refinement of system requirements and architecture while ensuring that major uncertainties are assessed gradually rather than all at once.

1.4.2 Why the Spiral Model

The Spiral Model presents several advantages that make it the optimal choice for our POD project:

- **Risk management:** The complex nature of a print-on-demand marketplace, particularly concerning localized payment integration, image processing for design mockups, and marketplace commission logic, introduces uncertainties that are best addressed through early risk assessment and prototyping.
- **Iterative refinement:** The model enables gradual development through multiple iterations, allowing requirements to be refined based on findings from previous cycles and feedback from supervisors and stakeholders.
- **Support for evolving requirements:** Certain aspects of the platform, such as designer dashboard features or storefront customization options, may require further exploration; the Spiral Model supports such evolving functional needs.
- **Progressive validation:** Each spiral delivers a prototype or functional module, making it possible to evaluate feasibility, usability, and performance progressively before full-scale implementation.

Because Printymand involves technical and business uncertainties, the Spiral Model lets us validate high-risk parts early and adapt as requirements become clearer. Initial spirals will prototype payment connectors and mockup generation to expose integration and capacity limits; later spirals will refine storefront features, scalability and deployment. This incremental risk-driven approach reduces late redesigns and focuses effort where it is most needed.

1.4.3 Project management with the Spiral Model

To understand how the Spiral Model will be implemented in our project, it is essential to define its core activities and how they apply to the development of the Printymand platform.

1.4.3.1 Project planning with the Spiral Model

Each spiral iteration consists of the following phases:

- **Objective determination and requirements identification:** The goals of the cycle are defined, along with the features or functionalities targeted in that phase. For instance, early iterations may focus on platform structure and user navigation, while later ones target core marketplace functionalities.
- **Risk analysis and resolution:** Potential risks such as payment gateway incompatibility, scalability concerns related to design uploads, or feasibility of real-time mockup generation are identified and assessed. Solutions may include prototyping, technical research, or consultation.
- **Development and validation:** A prototype or partial implementation is developed according to the cycle's objectives and then tested to validate its technical and functional correctness.
- **Evaluation and planning:** The results of the cycle are reviewed with the stakeholders (supervisors and project partners), and conclusions are used to plan the next spiral iteration.

1.4.3.2 Team and roles

The successful application of the Spiral Model requires clear responsibilities within the team. The roles involved in this project are defined in Table 1.1.

Table 1.1: Roles and responsibilities in the Spiral Model.

Role	**Mission**	**Actor**
Supervisor	Provides guidance, reviews progress at the end of each spiral, and advises on risk mitigation and methodology adjustments.	Mr./Mrs. [Supervisor Name]
Student Developer	Conducts planning, performs risk analysis, develops components, validates prototypes, and iteratively refines the project throughout each spiral.	Mohamed Amine Zini

Conclusion

In this chapter, we have presented an overview of the project's scope and objectives, examined the current state of POD solutions, pinpointed existing limitations, and proposed our innovative solution. Additionally, we have introduced the Spiral Model that will steer our project management through risk-driven, iterative cycles. In the chapters ahead, we will explore the architecture and planning stages, where we will specify the technical requirements and design elements essential for realizing our POD marketplace.

Chapter 2

Architecture and planning

2.1 Introduction

This chapter presents the architectural foundations and planning approach used to develop *printymand*. It starts with a requirements specification (functional and non-functional requirements), then introduces the system modeling artifacts (UML static and dynamic diagrams), and finally summarizes the implementation roadmap and the Spiral cycle planning that guided the iterative development.

2.2 Requirements specification

2.2.1 Functional requirements

Functional requirements describe the expected capabilities of the platform from the perspective of its primary roles (Customer, Designer, Printer, and Admin).

1 - Authentication & access control

- The system should allow users to register and log in securely.
- The system should enforce role-based access control (Customer, Designer, Printer, Admin).
- The system should manage sessions using JWT and protect sensitive endpoints.

2 - Catalog browsing & search

- The system should allow customers to browse available designs and products.
- The system should provide search and filtering capabilities to help users find relevant designs.
- The system should display design details and product information.

3 - Product customization

- The system should allow customers to customize products (size, color, placement).
- The system should provide a visual preview before order confirmation.
- The system should validate that a design is compatible with the selected product configuration.

4 - Printer discovery & selection

- The system should allow customers to compare printers by price, rating, processing time, delivery zone, and availability.
- The system should allow selecting a printer for a specific order based on constraints.

5 - Checkout & payment

- The system should display an order summary (product, design, printer, total cost).
- The system should allow customers to enter shipping information.
- The system should allow customers to select a payment method (Paymee, D17, Card).
- The system should confirm and place the order, persisting payment and invoice records.

6 - Order tracking & feedback

- The system should support order status progression: Pending → Confirmed → Printing → Shipped → Delivered.
- The system should notify users on relevant status updates.
- The system should provide order history and invoices.
- The system should allow customers to rate and review both the design and the printer after delivery.

7 - Designer functionalities

- The system should allow designers to create and manage a designer profile.
- The system should allow artwork upload with metadata and product mockup generation.
- The system should track royalties per sale and allow payout requests after reaching a threshold.
- The system should provide design performance metrics (views, favorites, sales) and automatic designer levels.

8 - Printer functionalities

- The system should allow printers to configure supported products, printing methods, base prices, processing times, and delivery zones.

- The system should support subscription plans that affect visibility and order capacity.
- The system should allow printers to accept/reject orders and update statuses.
- The system should allow printers to upload shipping and tracking information.

9 - Admin functionalities

- The system should allow administrators to manage users, verify accounts, and moderate designs.
- The system should allow monitoring of transactions, payouts, and subscriptions.
- The system should provide system-wide analytics and reports.

2.2.2 Non-functional requirements

Non-functional requirements describe quality attributes and constraints that affect the overall user experience and operational behavior of the platform. They cover aspects such as performance, usability, reliability, scalability, and security, ensuring that *printymand* remains trustworthy and efficient as usage grows.

1 - Performance

- The system should provide responsive browsing and search for designs and products, even under concurrent user load.
- The system should keep checkout and order placement operations efficient by minimizing unnecessary client–server round trips.
- The system should ensure that administrative dashboards and analytics queries remain usable by applying pagination and efficient database queries.

2 - Usability

- The UI should be consistent and easy to navigate for the four roles (Customer, Designer, Printer, Admin), with clear separation of capabilities.
- Users should be able to discover designs, customize products, select printers, and complete checkout with minimal friction.
- The system should provide clear feedback for key actions such as payment confirmation, order status changes, and upload validation.

3 - Reliability

- The platform should preserve order integrity by ensuring consistent status transitions and by preventing duplicated payments or duplicated orders.

- The system should remain available for core flows (authentication, browsing, checkout, tracking) and handle partial failures gracefully.

4 - Scalability

- The system should scale to accommodate growth in users, designs, and orders by using efficient database modeling and stateless REST services.
- The system should support horizontal scaling of the API layer and safe concurrent access to shared resources (orders, inventory constraints, payouts).

5 - Security

- The system should protect authentication and authorization using JWT and role-based access control (RBAC).
- The system should ensure secure storage and transmission of user data and payment-related metadata.
- Access to administrative and financial operations (moderation, subscriptions, payouts) should be restricted, audited, and monitored.

2.3 Global analysis

In this section, we take a closer look at the overall system design of *printymand*. We present static diagrams (use case and class diagrams) that capture the main actors and domain entities, as well as dynamic diagrams (sequence diagrams) that describe the interaction flow between the frontend, backend services, and the database. Finally, we provide an implementation roadmap and the Spiral cycle planning used to guide the iterative development of the platform.

2.3.1 Static diagrams

A static diagram portrays the fixed characteristics or attributes of a system or its components without showing any temporal progression.

2.3.1.1 Global use case diagram

The use case diagram is a visual tool that shows how users interact with the entire system, highlighting its main features. It helps to quickly understand what users want to achieve and how they interact with the system.

The figure 2.1 represents the global use case diagram .

2.3.1.2 Global class diagram

The class diagram is a visual representation of the structure of the system, showing the classes, attributes, and relationships between them. It helps to understand the organization of the system's components and their interactions.

The figure 2.2 represents the global class diagram .

2.3.1.3 Role-specific use case diagrams

In addition to the global diagrams, the following diagrams detail the specific interactions for each role in the *printymand* marketplace.

Customer use case diagram The customer use case diagram illustrates the primary interactions available to customers, including browsing designs, customizing products, selecting printers, completing checkout, and tracking orders.

Designer use case diagram The designer use case diagram shows the capabilities available to designers, such as creating and managing design profiles, uploading artwork, tracking performance metrics, and managing payouts.

Printer use case diagram The printer use case diagram depicts the operational workflows available to printers, including configuring supported products, managing pricing and processing times, accepting/rejecting orders, and updating order statuses.

Admin use case diagram The admin use case diagram represents the administrative capabilities, such as managing users, verifying accounts, moderating designs, monitoring transactions, and analyzing platform metrics.

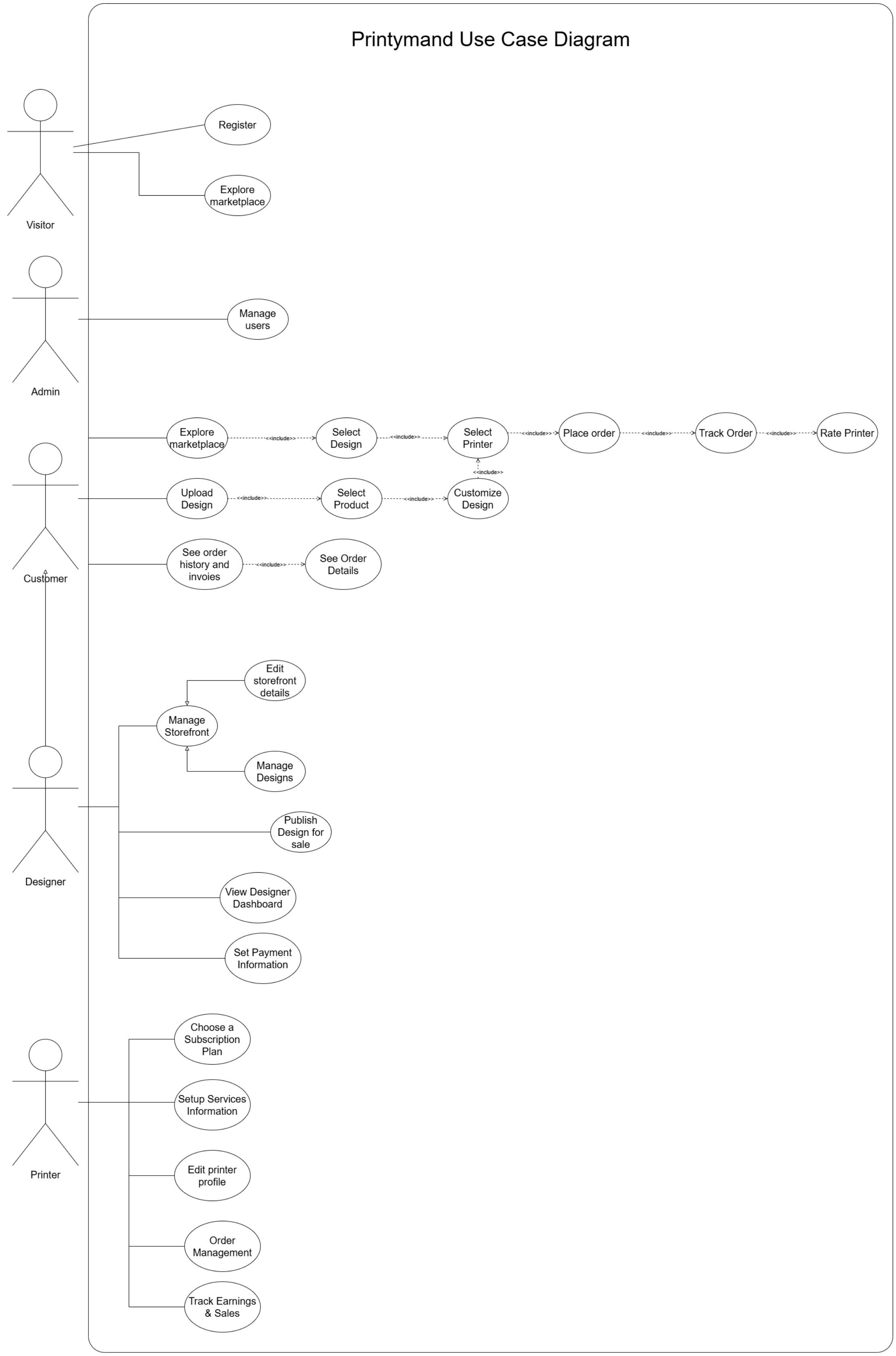

Figure 2.1: Global use case diagram.

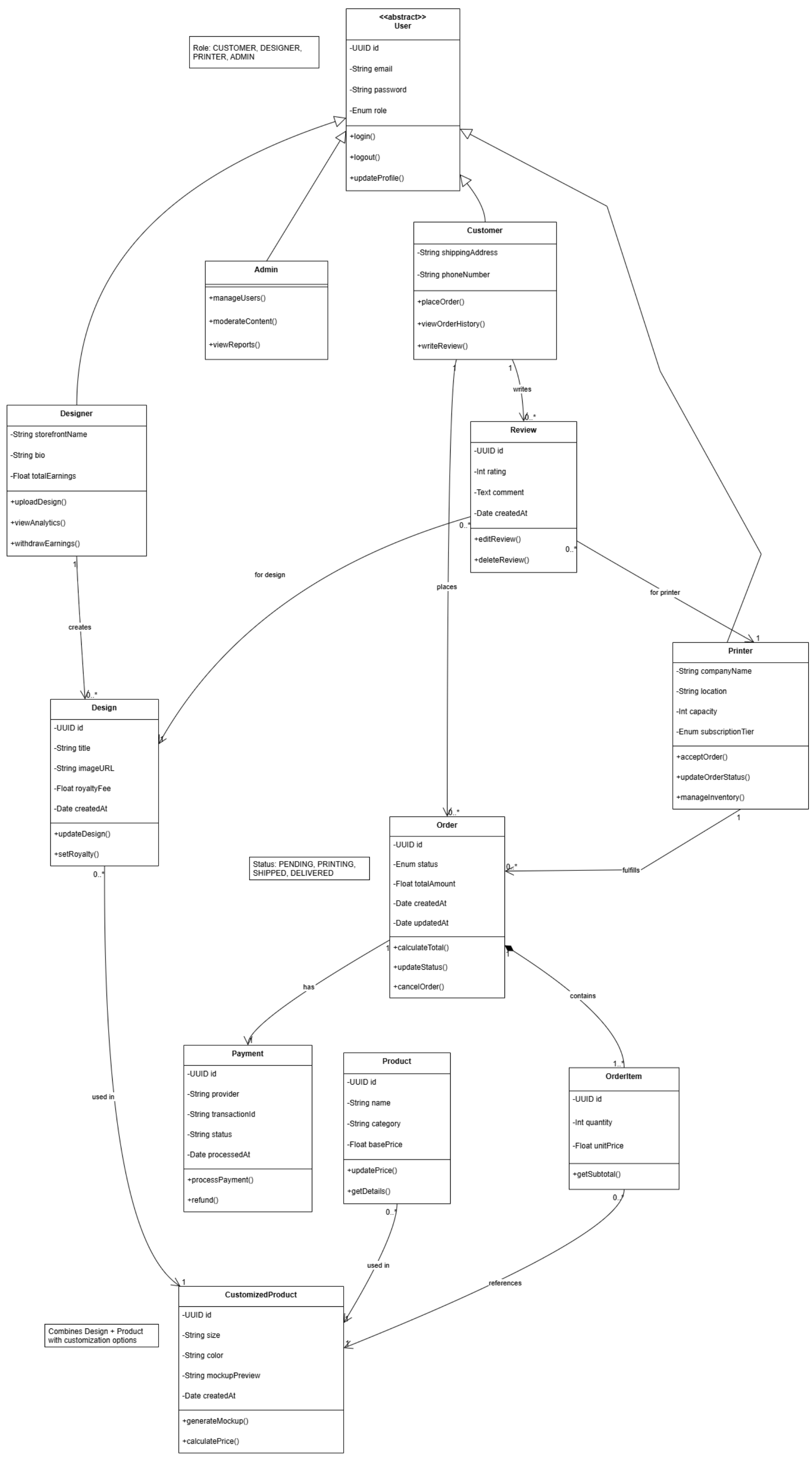

Figure 2.2: Global Class Diagram

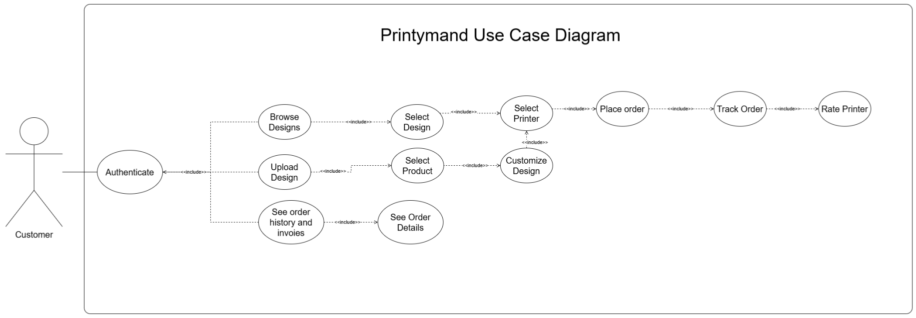

Figure 2.3: Customer use case diagram.

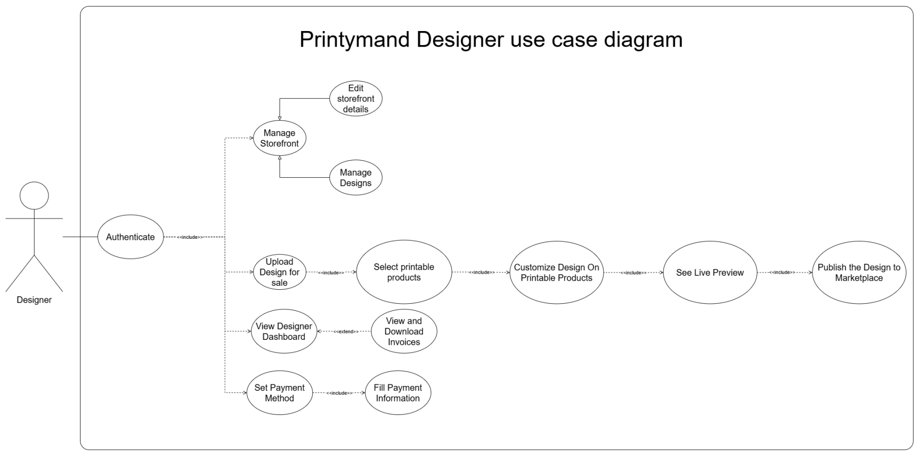

Figure 2.4: Designer use case diagram.

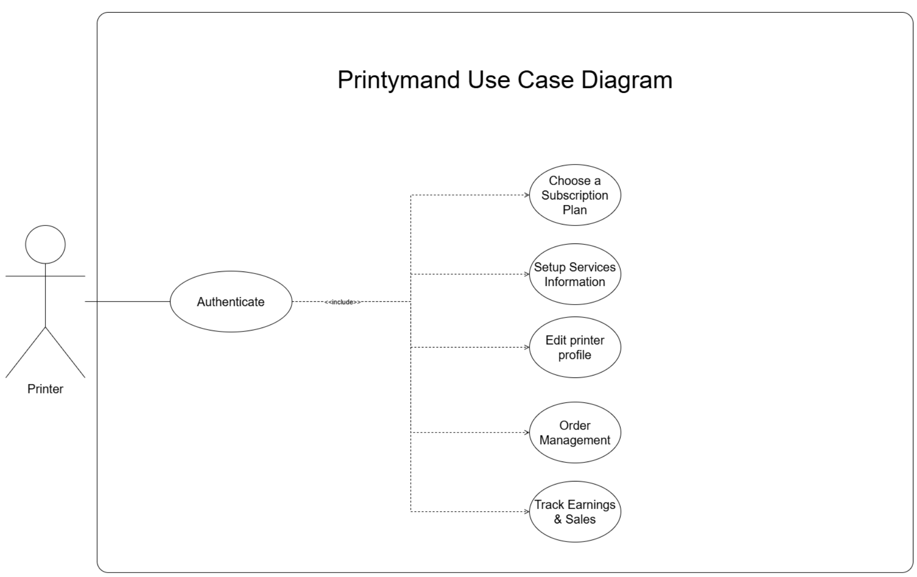

Figure 2.5: Printer use case diagram.

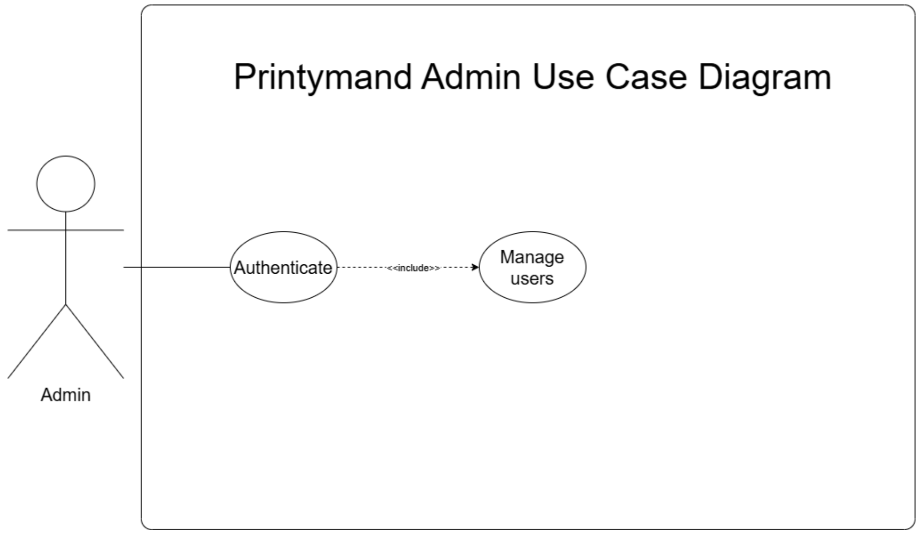

Figure 2.6: Admin use case diagram.

2.3.2 Dynamic diagrams

A dynamic diagram illustrates how a system evolves and behaves over time, capturing its operations, actions, and changes.

2.3.2.1 Global sequence diagram

The sequence diagram provides a visual representation of how different components of the system interact over time, illustrating the flow of messages between them. It helps to understand the chronological order of actions and the dependencies between system components. We have introduced global sequence diagrams from both the user and admin perspectives.

The user's global sequence diagram highlights how each action is processed by the system entities, including the user interface, the *printymand* API, and the database. It covers a typical end-to-end marketplace workflow (authentication, browsing a design, customizing a product, selecting a printer, checkout, and order tracking) with alternative scenarios such as validation errors, payment failure, or unavailable printer capacity. As shown in the figure 2.3 below.

Figure 2.7: Printymand global sequence diagram.

2.3.2.2 Design Upload & Listing sequence diagram

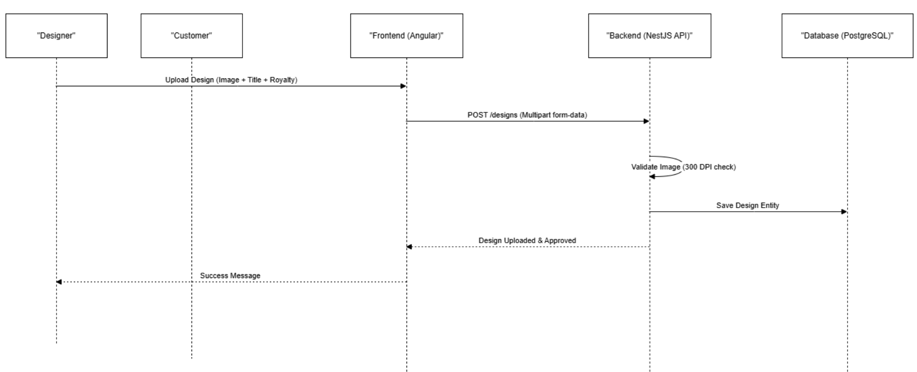

Figure 2.8: Design Upload & Listing sequence diagram.

2.3.2.3 Browsing & Customization sequence diagram

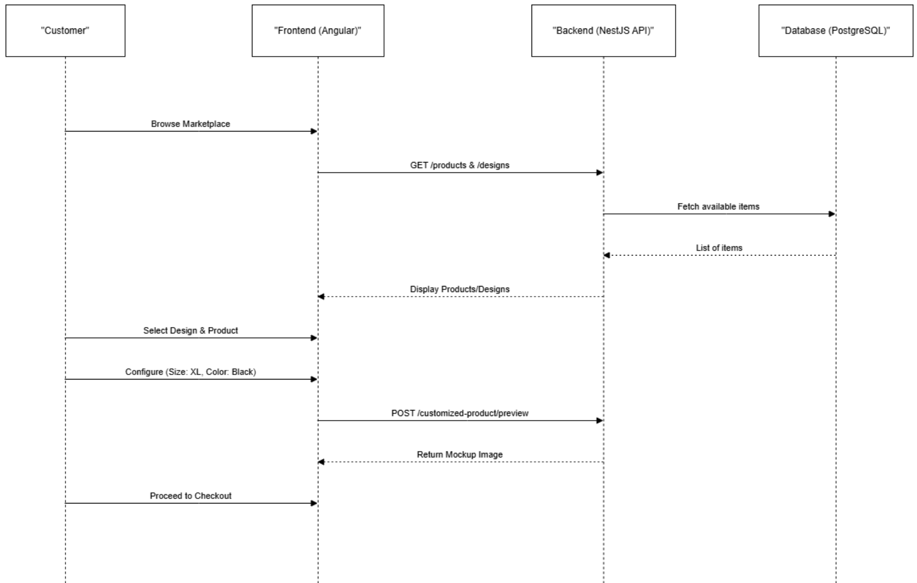

Figure 2.9: Browsing & Customization sequence diagram.

2.3.2.4 Selection & Checkout sequence diagram

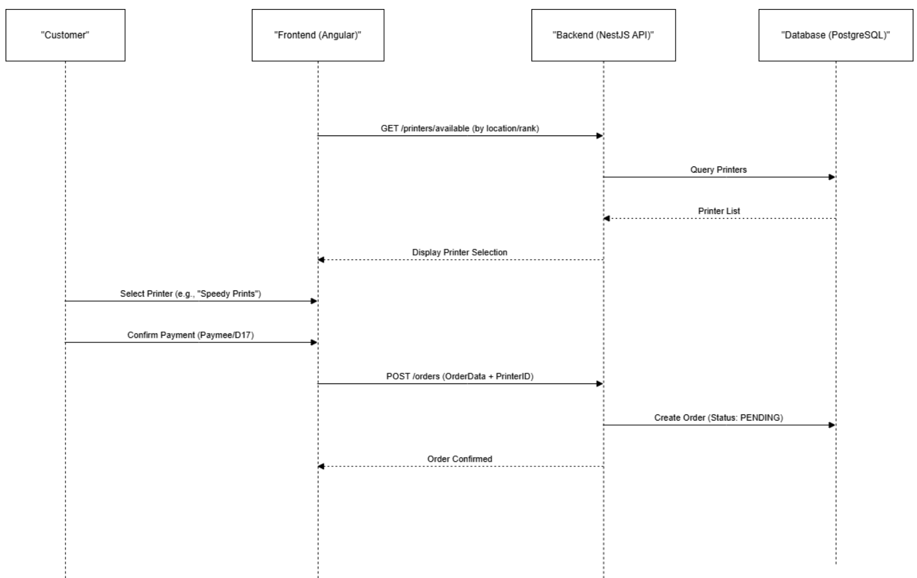

Figure 2.10: Selection & Checkout sequence diagram.

2.3.2.5 Fulfillment & Delivery sequence diagram

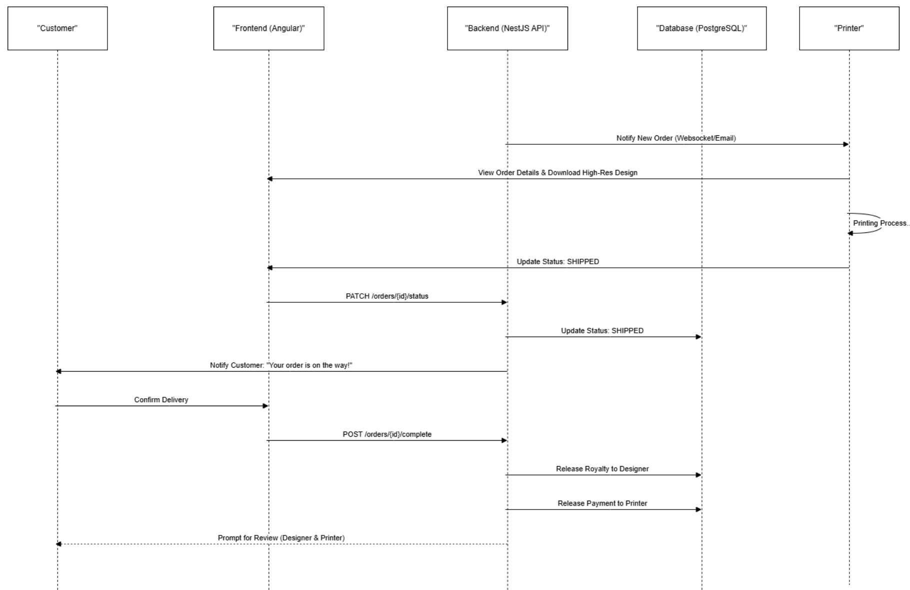

Figure 2.11: Fulfillment & Delivery sequence diagram.

2.3.3 Implementation roadmap

This section outlines a high-level implementation roadmap: a structured list of functionalities that need to be completed to achieve the project's goals. Each line contains a capability with its ID, a short description, priority, and complexity.

The table 2.1 represents the global implementation roadmap of the project.

Table 2.1: Implementation roadmap.

ID	Capability	Description	Priority	Complexity
1	Authentication & RBAC	Secure registration/login and role-based access control across Customer, Designer, Printer, and Admin operations.	High	Medium
2	Design catalog & search	Customer-facing catalog browsing and design discovery with search and filtering capabilities.	High	Medium
3	Product customization	Product configuration (size, color, placement) with preview before order confirmation.	High	High
4	Printer selection	Printer comparison and selection using price, rating, processing time, delivery zone, and availability constraints.	High	Medium
5	Checkout & payment	Checkout flow with shipping details, payment confirmation, and persisted order records.	High	High

ID	Capability	Description	Priority	Complexity
6	Order lifecycle & tracking	Order status progression and tracking with access to order history and invoices.	High	Medium
7	Designer upload & royalties	Design upload management and royalty tracking per sale to support designer monetization.	High	High
8	Printer dashboard	Printer operational views for order processing, pricing/capability management, and status updates.	High	High
9	Reviews & ratings	Post-delivery ratings and reviews for designs and printers to support trust and discovery.	Medium	Medium
10	Subscriptions	Subscription plans affecting printer visibility and capacity on the platform.	Medium	Medium
11	Administration & moderation	Administrative tools for user/role management, content moderation, and transaction monitoring.	High	High

2.3.4 Spiral cycle planning

The Spiral Model structures development into successive cycles. Each cycle starts by defining objectives and constraints, then performs risk analysis, followed by design and implementation, and ends with evaluation and planning for the next cycle. The planning below summarizes how the implementation roadmap was organized into three risk-driven cycles, each delivering a usable increment.

Cycle 1 (Week 1–4)

Description

Cycle 1 focuses on establishing the technical foundation of *printymand* while reducing early risks related to security and asset handling. It delivers authentication with RBAC, the initial customer-facing catalog experience, and a baseline design upload pipeline.

Objectives

1. Implement authentication and role-based access control;
2. Create the marketplace catalog foundation (designs and products);
3. Design and develop the initial Angular UI structure;
4. Set up the backend API foundations and database schema.

Validation focus

1. Implement registration, login, and JWT issuance with secure password handling;
2. Implement RBAC guards for Customer, Designer, Printer, and Admin operations;
3. Build initial catalog endpoints and pages (list, details, basic filtering);

4. Define the core database entities (users, roles, designs, products, printers).

Cycle 2 (Week 5–8)

Description

Cycle 2 implements the end-to-end transaction flow of the marketplace while addressing risks around order integrity and payment confirmation. This includes product customization, printer selection, checkout, and order lifecycle tracking.

Objectives

1. Implement product customization and preview;
2. Implement printer discovery and selection;
3. Implement checkout and payment integration;
4. Implement order lifecycle management and tracking.

Validation focus

1. Implement customization logic and persist customer configuration in the order;
2. Implement printer filtering and comparison based on price, rating, and delivery constraints;
3. Implement checkout flow and store payment metadata and invoices;
4. Implement order status transitions and notifications for key events.

Cycle 3 (Week 9–12)

Description

Cycle 3 consolidates the platform for operational use by implementing governance features, improving performance, and producing exports/invoices and documentation. Integration validation confirms end-to-end marketplace workflows and administrative control paths.

Objectives

1. Implement designer and printer dashboards (uploads, orders, analytics);
2. Implement subscriptions, royalties, and payout-related flows;
3. Implement admin moderation and monitoring features.

Validation focus

1. Implement designer management (upload, performance metrics, royalty tracking);
2. Implement printer management (plans, capacity, order processing and tracking input);
3. Implement admin tooling (user management, design moderation, transaction monitoring).

2.4 Work environment

In this section, we will outline the different aspects of our work environment that play an important role in the development and success of our project. This includes understanding the modeling language as well as the hardware and software environments that enable us to deliver a high-quality product.

2.4.1 Software architectural pattern

The *printymand* platform follows a layered web architecture that separates concerns between the presentation layer, the application layer, and the data layer. This separation is consistent with the general intent of patterns such as MTV/MVC, where responsibilities are clearly divided to improve maintainability and scalability.
The presentation layer is implemented as an Angular single-page application, responsible for user interaction, form validation, and rendering role-specific dashboards.
The application layer is implemented with a NestJS REST API, where controllers expose endpoints, services encapsulate business logic (orders, payments, subscriptions, royalties), and guards enforce authentication and RBAC.
The data layer is provided by PostgreSQL, which stores persistent entities such as users, designs, printers, orders, payments, and reviews. The figure 2.5 shows the MTV infographic.

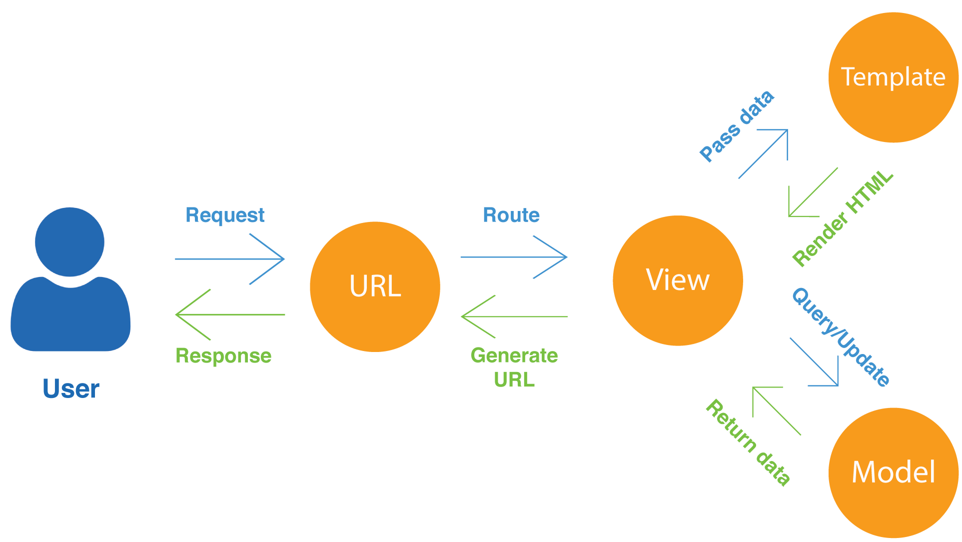

Figure 2.12: MTV infographic.

2.4.2 Modeling language

In our project, we employed the Unified Modeling Language (UML) to visualize and design the system architecture. UML is a widely recognized and standardized modeling language known for its effectiveness in representing complex systems. We selected UML because it enhances communication among team members and involved parties through clear and detailed diagrams. These diagrams help us map out system components, their interactions, and the overall workflow, enabling a detailed understanding of the project's structure. By utilizing UML, we can identify potential issues early in the development process, ultimately contributing to a higher quality final product. The figure 2.6 shows the UML logo.

Figure 2.13: UML logo.[1]

2.4.3 Hardware environment

The hardware environment sets the stage for the performance and reliability of our development activities. Here, we will detail the hardware specifications and configurations that support our development process.

The table 2.2 represents the hardware environment details .

Table 2.2: Hardware environment table.

Brand	Dell G15 5530	HP Laptop 15-dw3xxx
CPU	Intel® Core™ i5-13450HX	Intel® Core™ i5-1135G7
Installed RAM	16GB	12GB
Operating system	Windows 11	Windows 11
GPU	NVIDIA GeForce RTX 3050	NVIDIA GeForce MX350

2.4.4 Software environment

A strong software environment is essential for effective development and collaboration. This subsection presents the software tools, frameworks, and programming languages that were used to build the *printymand* marketplace platform.

2.4.4.1 Development tools

Development tools are critical in aiding our coding, testing, and debugging efforts. In this subsection, we will describe the specific tools we employ to support our development process.

The table 2.4 shows the different development tools used in this project .

Table 2.3: Development tools table.

Logo	Description
	VS Code: Visual Studio Code is a lightweight but powerful source code editor available for Windows, macOS, and Linux. It comes with built-in support for JavaScript, TypeScript, and Node.js and has a rich ecosystem of extensions for other languages and runtimes.[2] **Usage in Project:** VS Code was utilized as the primary code editor for writing and managing the project's codebase and the LaTeX editor for the report due to its efficiency and extensive features.
	Git: Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.[3] **Usage in Project:** Git was employed for version control, ensuring efficient tracking and management of code changes throughout the project's development.
	GitHub: GitHub is a cloud-based platform where you can store, share, and work together with others to write code.[4] **Usage in Project:** GitHub was used in conjunction with Git for version control and collaboration, hosting the project's repository and facilitating teamwork.

Logo	Description
	Adobe Illustrator: Adobe Illustrator is a vector graphics editor and design program developed and marketed by Adobe Inc. It is used by graphic designers to create vector images, illustrations, and artwork for various media.[5] **Usage in Project:** Adobe Illustrator was employed to create graphic illustrations for the project, enhancing the visual appeal and clarity of some graphical elements.
	Notion: Notion is a freemium productivity and note-taking web application developed by Notion Labs, Inc. It offers organizational tools including task management, project tracking, to-do lists, and bookmarking.[6] **Usage in Project:** Notion was used for project management, tracking project progress and organizing tasks.

2.4.4.2 Libraries, frameworks and programming languages

Libraries, frameworks and programming languages form the foundation of our software development efforts. This subsection will provide an overview of the frameworks and languages we use.

Table 2.4: Libraries, frameworks and pogramming languages table.

Logo	Description
	Angular: Angular is a TypeScript-based web application framework used to build single-page applications with a component-based architecture.[7] **Usage in Project:** Angular was used to implement the *printymand* frontend, including role-based views, catalog pages, and checkout flows.

Logo	Description
	NestJS: NestJS is a progressive Node.js framework for building efficient and scalable server-side applications, commonly used to implement REST APIs with a modular structure.[8] **Usage in Project:** NestJS was used to implement the *printymand* backend API, including authentication, RBAC, order management, and administrative operations.
	HTML: HTML is the standard markup language for documents designed to be displayed in a web browser. It defines the content and structure of web content. It is often assisted by technologies such as Cascading StyleSheets (CSS) and scripting languages such as JavaScript.[9] **Usage in Project:** HTML was used within Angular templates to structure the pages of the application and present the marketplace content.
	CSS: CSS is a stylesheet language used to describe the presentation of a document written in HTML or XML. CSS describes how elements should be rendered on screen, on paper, in speech, or on other media.[10] **Usage in Project:** CSS was used to style Angular components and ensure a consistent, responsive interface across the customer, designer, printer, and admin views.
	TypeScript: TypeScript is a lightweight interpreted (or just-in-time compiled) programming language with first-class functions.[11] **Usage in Project:** TypeScript (a typed superset of JavaScript) was used as the primary language for both the Angular frontend and the NestJS backend.
	Tailwind CSS: Tailwind CSS is a utility-first CSS framework that provides low-level classes to build custom designs directly in markup.[12] **Usage in Project:** Tailwind CSS was used to accelerate UI implementation and keep styling consistent across the *printymand* Angular application.

Logo	Description
	PostgreSQL: PostgreSQL is an advanced open-source relational database management system, known for reliability and strong support for transactional workloads.[13] **Usage in Project:** PostgreSQL was used as the main database for *printymand*, persisting users, designs, printers, orders, payments, and reviews.
	ORM (Prisma/TypeORM): An Object-Relational Mapping (ORM) layer maps database tables to application models, simplifying persistence operations and enforcing consistency.[14] **Usage in Project:** An ORM was used to access PostgreSQL from the NestJS backend in a structured way, reducing boilerplate SQL and improving maintainability.

2.5 Conclusion

In conclusion, the architecture and planning phase laid the foundation for the implementation of the *printymand* project. In the next chapters, we present the Spiral-cycle implementation, illustrating how the planned features were delivered through risk-driven development cycles.

Chapter 3

Spiral Cycle 1: Authentication, Onboarding, and Marketplace Discovery

3.1 Introduction

This chapter describes the first Spiral Model cycle of the *printymand* project. The objective of Cycle 1 is to deliver a coherent and usable entry point to the platform by implementing authentication and onboarding, and by providing the first customer-facing browsing experience. The delivered increment includes the login and registration interfaces, role selection after sign-up, the marketplace exploration page (search, filters, and design listing), and a product page that presents item details, basic configuration options, and reviews.

3.2 Cycle planning and objectives

In the Spiral Model, each cycle starts by defining concrete objectives and acceptance criteria for the increment, which guides engineering work and validation activities. Table 3.1 summarizes the main objectives and deliverables of Cycle 1.

3.3 Risk analysis

Because the Spiral Model is risk-driven, Cycle 1 focuses on reducing uncertainties that could compromise later cycles. The main risks and mitigation actions addressed are summarized below.

3.3.1 Security and access separation

Risk: authentication flaws or weak access separation can expose role-specific views and undermine user trust.

Table 3.1: Cycle 1 objectives and deliverables.

ID	Capability	Objective (expected outcome)	Priority	Complexity
1	Authentication & onboarding	Provide login and registration interfaces with basic session entry points and user onboarding screens.	High	Medium
2	Role selection	Allow users to select how they will use the platform (Customer, Designer, Printer, Admin) and navigate to role-specific dashboards.	High	Low
3	Marketplace exploration	Provide a marketplace page to discover designs (search, filters, sorting, and design grid).	High	Medium
4	Product page & reviews	Provide a product page with details, configuration options (type/color/size/quantity), and a reviews section.	High	Medium

Mitigation: implement secure authentication flows and consistent access checks for role navigation, and provide clear error handling for unauthorized requests.

3.3.2 Onboarding drop-off and usability

Risk: complex onboarding (registration, role selection, login) can increase drop-off and reduce adoption.
Mitigation: keep onboarding screens focused, provide clear calls to action, and ensure consistent navigation between login, registration, and role selection.

3.3.3 Catalog consistency and discovery quality

Risk: inconsistent data presentation (cards, ratings, prices, categories) reduces marketplace credibility and makes discovery harder.
Mitigation: standardize how marketplace items are rendered (card layout, metadata, ratings) and ensure that filters/sorting behave predictably.

3.3.4 Performance under browsing load

Risk: marketplace browsing involves many cards and images; slow loading harms the perceived quality of the platform.
Mitigation: keep the marketplace UI responsive (pagination and lightweight cards) and avoid blocking interactions during filtering and sorting.

3.4 Validation and evaluation

In this section, we validate the delivered objectives of Cycle 1 by describing how the main UI features behave and how they support the marketplace entry flow.

3.4.1 Authentication and onboarding flow

Cycle 1 provides the entry screens of *printymand*: login and registration. Registration leads to a role selection screen where users choose their intended usage (Customer, Designer, Printer, or Admin) before being routed to role-specific dashboards. This onboarding flow provides a clear separation between the marketplace browsing experience and administrative/operational views.

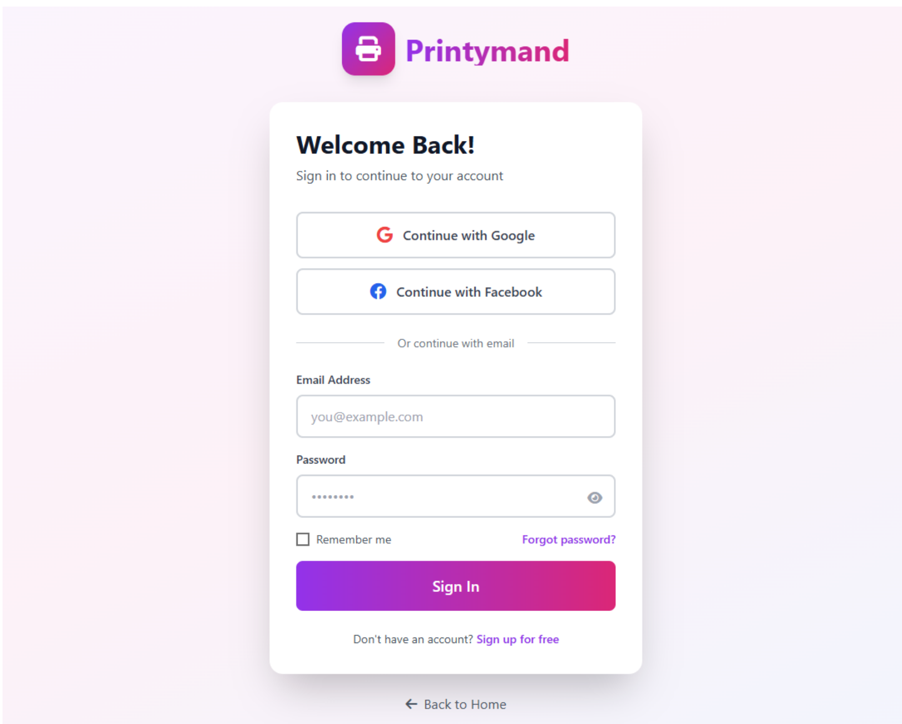

Figure 3.1: Cycle 1: Authentication Signin interface.

3.4.2 Marketplace exploration

The marketplace page supports discovery by presenting a grid of designs and providing interactive controls such as search, filters, and sorting. This enables customers to find relevant designs efficiently and to access a product page from the listing.

3.4.3 Product page and configuration widgets

The product page presents detailed information about a selected item and includes configuration widgets such as product type, color, size, and quantity. A reviews section summarizes

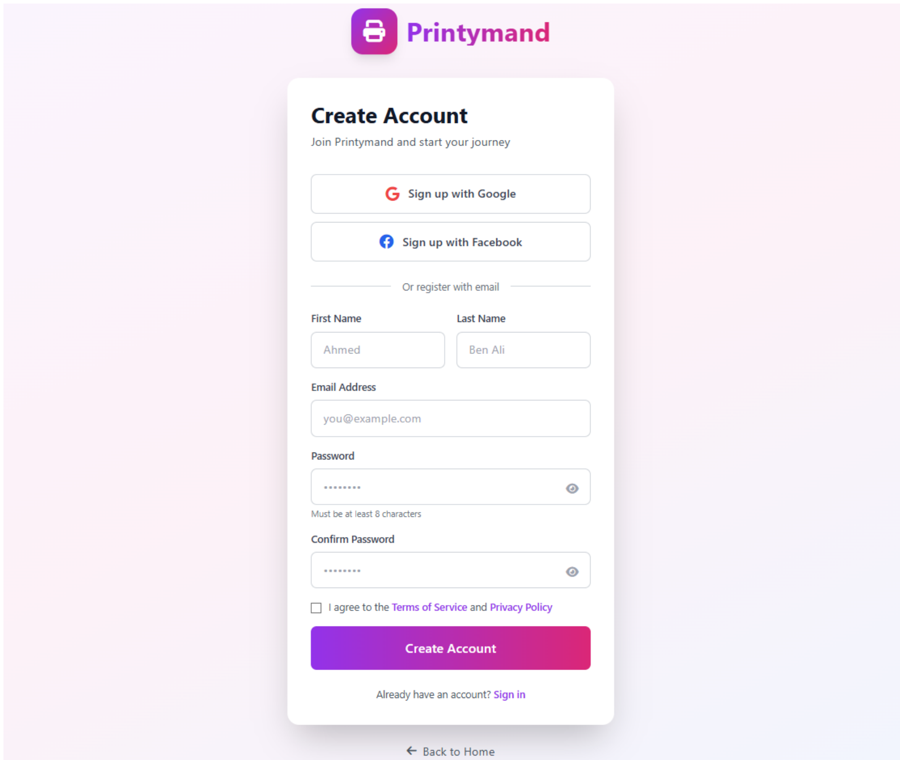

Figure 3.2: Cycle 1: Authentication Signup interface.

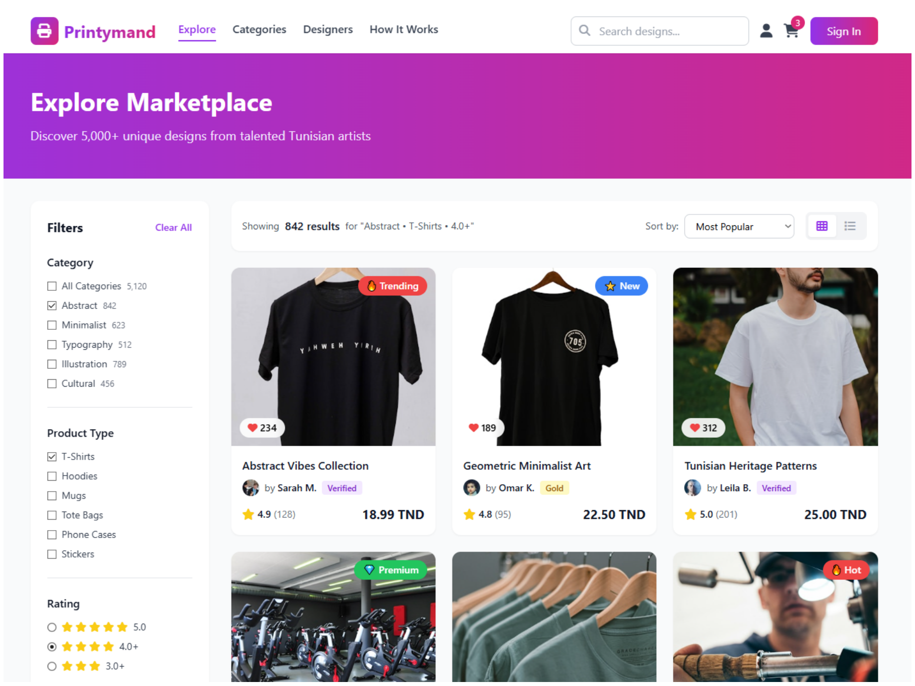

Figure 3.3: Cycle 1: Marketplace discovery interface.

user feedback and provides a structured list of individual reviews, helping users make informed purchase decisions.

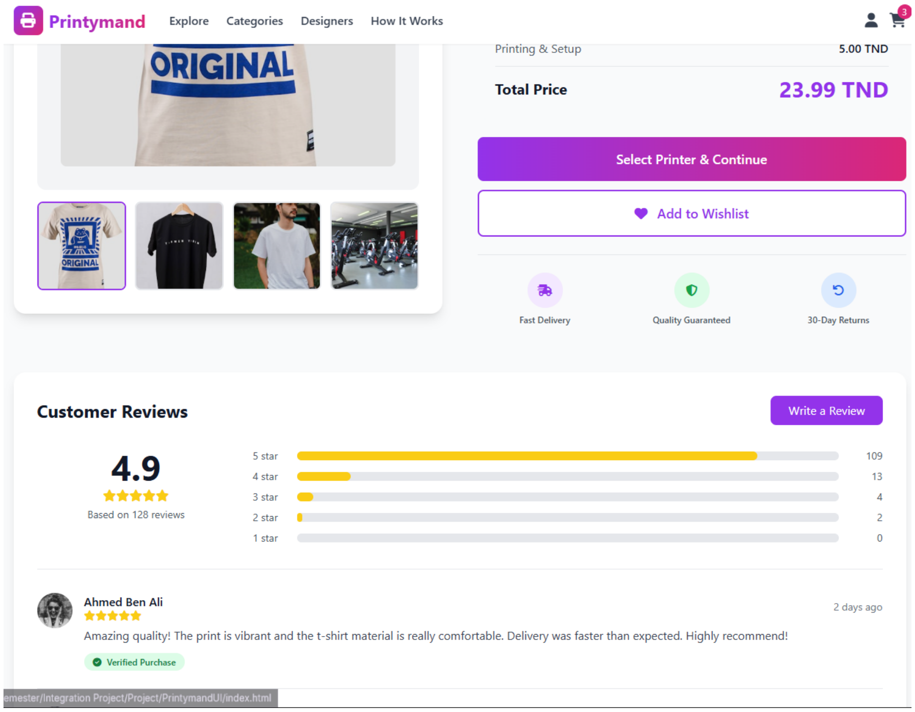

Figure 3.4: Cycle 1: Product page interface.

3.4.4 Interface coherence and navigation

Across Cycle 1 screens, the interface remains consistent in layout and navigation (top navigation, call-to-action buttons, and clear headings). This coherence reduces cognitive load and prepares users for the more complex transaction workflow introduced in Cycle 2.

3.5 Conclusion

In conclusion, Cycle 1 delivered the practical entry point of the *printymand* platform by implementing authentication and onboarding (including role selection), and by delivering the first customer-facing browsing experience (marketplace exploration and product details with reviews). This cycle reduces early usability and trust risks and provides a stable baseline for Cycle 2, which introduces the end-to-end purchase workflow.

Chapter 4

Spiral Cycle 2: Customization, Checkout, and Order Tracking

4.1 Introduction

This chapter describes the second Spiral Model cycle of the *printymand* project. Building on the foundation established in Cycle 1 (authentication, catalog browsing, and the design-asset pipeline), Cycle 2 delivers the core marketplace transaction workflow: product customization, printer selection, checkout/payment, and order tracking.

4.2 Cycle planning and objectives

In the Spiral Model, each cycle starts by defining what must be achieved and what evidence will be used to evaluate success. Table 4.1 summarizes the objectives and deliverables of Cycle 2.

4.3 Risk analysis

Cycle 2 focuses on risks related to the transaction workflow and data integrity, because these aspects directly affect marketplace trust.

4.3.1 Order integrity and status consistency

Risk: inconsistent state transitions can lead to duplicated orders, incorrect tracking, or broken printer workflows.
Mitigation: enforce clear status progression and persist a single authoritative order record that is updated through controlled transitions.

Table 4.1: Cycle 2 objectives and deliverables.

ID	Capability	Objective (expected outcome)	Priority	Complexity
1	Product customization	Allow customers to configure product options (size, color, placement) and confirm their choice through a preview before ordering.	High	High
2	Printer selection	Allow customers to compare and select printers using business and delivery constraints (price, rating, processing time, zone).	High	Medium
3	Checkout & payment	Persist a confirmed order after shipping details and payment confirmation, with clear association between customer, design, and printer.	High	High
4	Order tracking	Provide order lifecycle visibility through a tracking view with timeline/status milestones and a clear order summary.	High	High

4.3.2 Payment integration and failure handling

Risk: payment confirmation failures, timeouts, or mismatches between UI state and backend state can cause user confusion or invalid orders.
Mitigation: create the order only after successful confirmation and return explicit API responses for UI feedback.

4.3.3 Preview-to-production mismatch

Risk: if the customization preview does not reflect the stored configuration, the printed output may not match customer expectations.
Mitigation: persist the configuration as part of the order/cart record and reuse it consistently for later steps.

4.3.4 Printer selection reliability

Risk: selecting a printer without validated constraints (delivery zone, capacity, pricing) can create fulfillment delays or cancellations.
Mitigation: expose selection criteria clearly and ensure that printer choice is an explicit confirmation step before checkout.

4.4 Validation and evaluation

In this section, we validate the objectives of Cycle 2 by detailing the implemented workflow and presenting representative outputs (UI states and API responses). This evaluation demonstrates that the platform can support an end-to-end transaction.

4.4.0.1 Product configuration

The customer starts from the product page, where the interface provides configuration widgets such as product type, color selection, size selection, and quantity. The preview area and the configuration panel allow the customer to review the chosen options before proceeding. This step reduces ambiguity and prepares the selection for printer comparison.

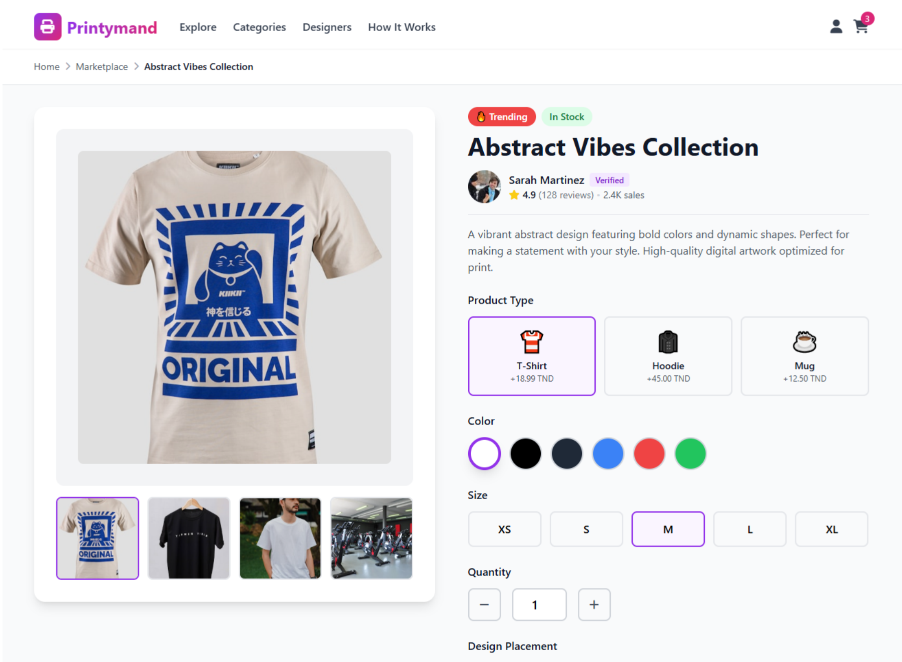

Figure 4.1: cycle 2 : product customization interface

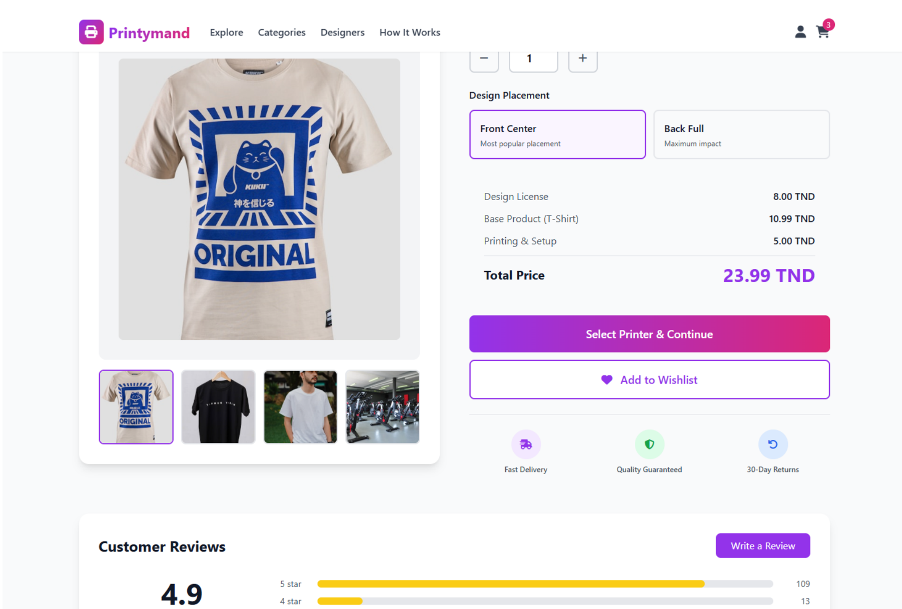

Figure 4.2: cycle 2 : product customization interface 2

4.4.0.2 Printer selection

After product configuration, the customer transitions to the printer selection step (stepper progress: Product → Select Printer → Checkout). Printers are presented as cards with decision criteria such as rating, price, processing time, and delivery constraints. The interface provides an order summary sidebar to keep the configured item visible while selecting a fulfillment partner.

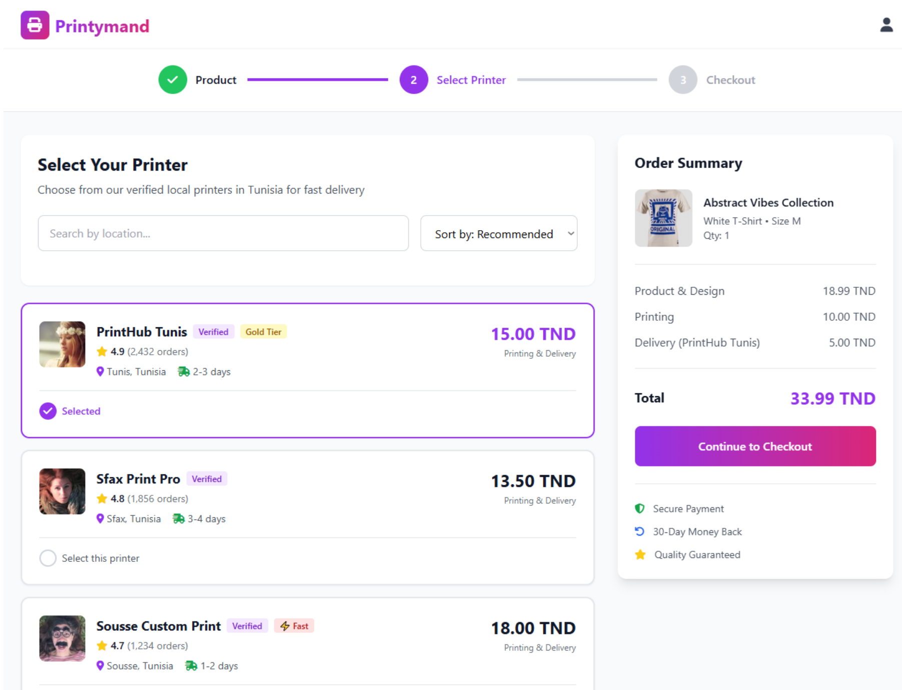

Figure 4.3: cycle 2 : printer selection interface

4.4.0.3 Checkout and payment

The checkout page collects contact information and delivery address details, then allows the customer to select a payment method. In the provided UI, payment options include Paymee, D17, and card payment. This step finalizes the order intent and transitions the user toward confirmation and tracking.

4.4.0.4 Order confirmation and tracking

Once the order is placed, the tracking page confirms success and displays a unique order identifier. It also provides a timeline of order status milestones (for example: Pending, Confirmed, Printing, Shipped, Delivered), an order details section, and a summary sidebar that helps the customer monitor progress and expected delivery.

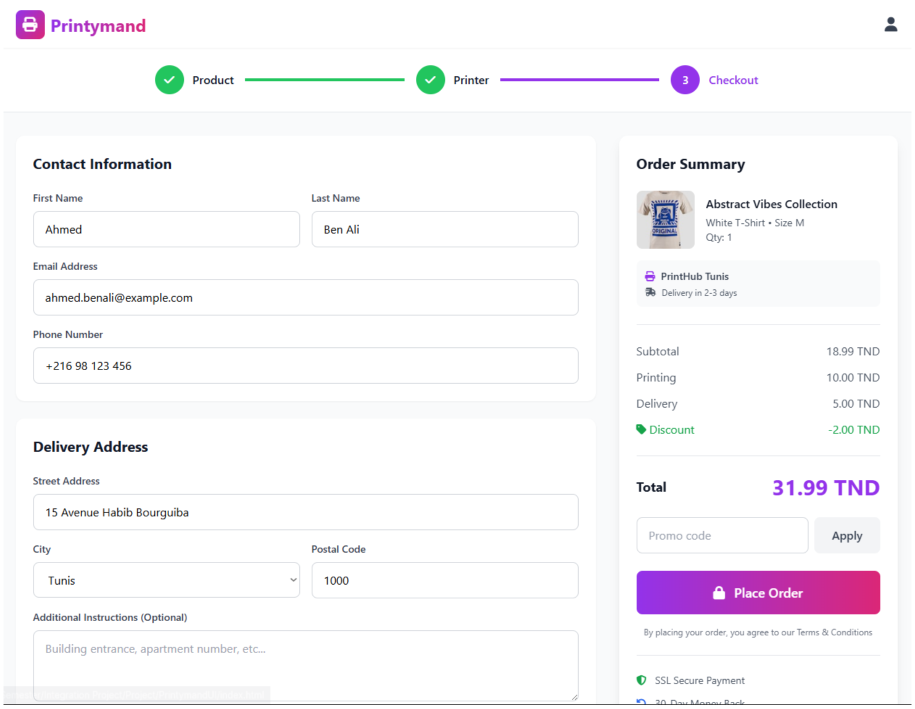

Figure 4.4: cycle 2 : checkout and payment interface 1

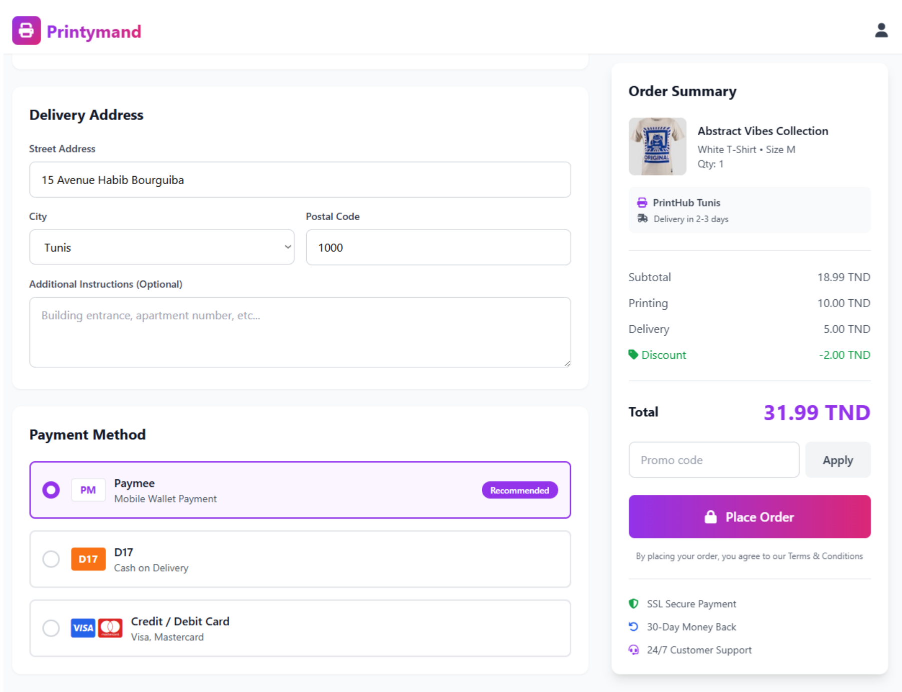

Figure 4.5: Cycle 2 : checkout and payment interface 2

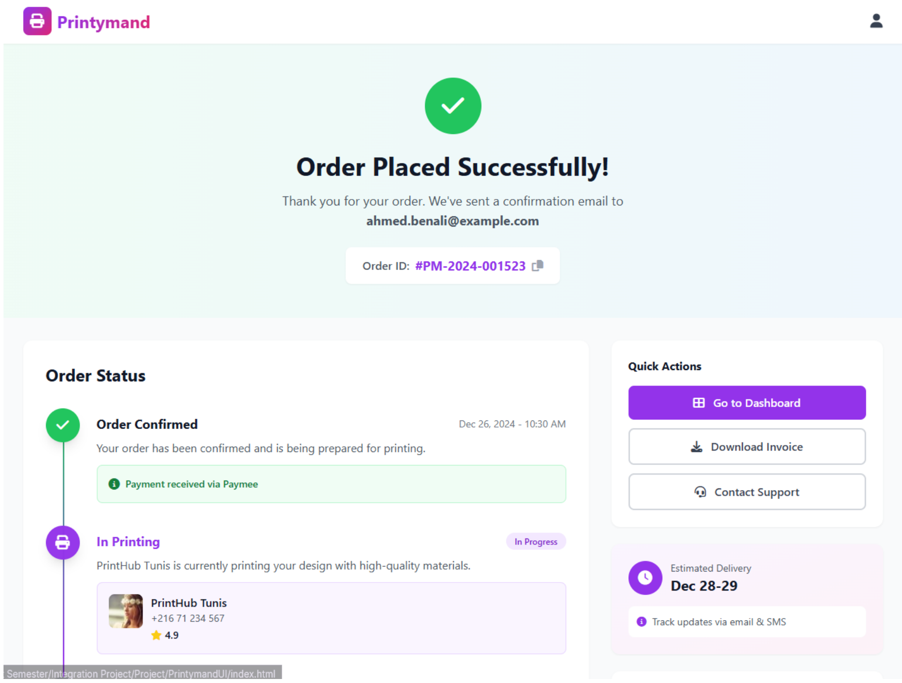

Figure 4.6: Cycle 2: Order tracking interface.

4.5 Conclusion

In conclusion, Cycle 2 delivered the core marketplace transaction workflow by implementing product customization, printer selection, checkout/order confirmation, and order tracking with invoice-related outputs. By addressing risks related to order integrity and payment confirmation, this cycle provides the operational baseline needed for Cycle 3, which strengthens governance capabilities, administrative control, and platform optimization.

Chapter 5

Spiral Cycle 3: Governance, Administration, and Platform Optimization

5.1 Introduction

This chapter describes the third Spiral Model cycle of the *printymand* project. After delivering the end-to-end purchase workflow in Cycle 2, Cycle 3 focuses on operational readiness through role-specific dashboards and administrative oversight. The delivered increment consolidates the platform around four roles (Customer, Designer, Printer, and Admin) by providing dedicated dashboards with clear metrics and operational actions.

5.2 Cycle planning and objectives

In the Spiral Model, later cycles often target quality, governance, and operational readiness once the core workflow is stable. Table 5.1 summarizes the main objectives and deliverables of Cycle 3.

5.3 Risk analysis

Cycle 3 reduces risks that typically appear when transitioning from an end-to-end workflow to a role-operational marketplace.

5.3.1 Governance and compliance

Risk: without administrative visibility, platform issues (misconfigured accounts, abnormal orders, operational exceptions) can remain unnoticed.
Mitigation: implement an admin dashboard that surfaces platform metrics and provides access to management actions.

Table 5.1: Cycle 3 objectives and deliverables.

ID	Capability	Objective (expected outcome)	Priority	Complexity
1	Customer dashboard	Provide customer visibility over orders (totals and status distribution) and quick access to active orders and wishlist indicators.	High	Medium
2	Designer dashboard	Provide designer visibility over published designs and earnings (active designs, sales, revenue, rating) with an entry point to upload/manage designs.	High	Medium
3	Printer dashboard	Provide printer operational visibility over incoming orders (new orders, in production) and revenue, with workflow actions to process and update orders.	High	High
4	Admin dashboard	Provide platform-wide monitoring (users, orders, revenue) and management entry points for administrative oversight.	High	High

5.3.2 Role separation and authorization drift

Risk: dashboards expose sensitive information (revenues, operational orders, platform statistics); incorrect role separation can leak data across roles.
Mitigation: enforce role-based routing and permissions consistently for Customer/Designer/Printer/Admin views.

5.3.3 Operational workflow consistency

Risk: printers need a reliable view of order status and workload; inconsistent updates can cause delays and user dissatisfaction.
Mitigation: provide a printer dashboard that highlights new orders and in-production orders and supports clear status tracking.

5.3.4 Maintainability and handover

Risk: without clear documentation, onboarding and future changes become risky and error-prone.
Mitigation: produce documentation describing architecture, API contracts, and operational workflows.

5.3.5 Implemented capabilities overview

5.3.5.1 Customer dashboard

The customer dashboard provides a concise overview of the customer state: total orders, delivered orders, in-transit orders, and wishlist indicators. It also serves as a starting point to review active orders and to access marketplace navigation.

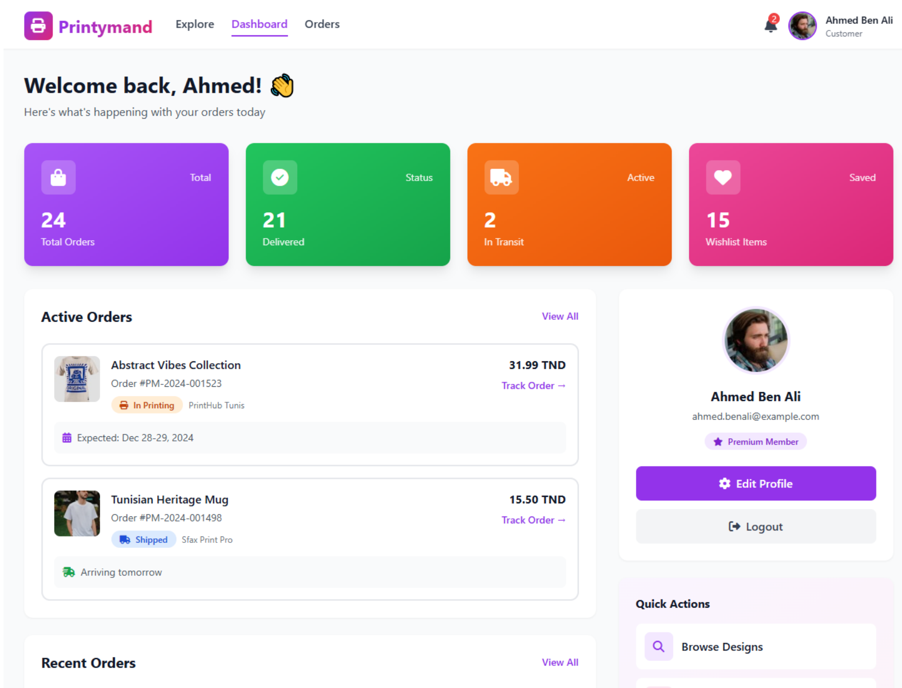

Figure 5.1: Cycle 3 : Customer dashboard interface

5.3.5.2 Designer dashboard

The designer dashboard focuses on design activity and earnings. It presents key metrics such as the number of active designs, total sales, total earnings (in TND), and rating. It also provides a clear call-to-action to upload or manage designs.

5.3.5.3 Printer dashboard

The printer dashboard is operational: it highlights workload (new orders and in-production orders), revenue, and average score. It supports daily actions such as monitoring orders and tracking fulfillment progress.

5.3.5.4 Admin dashboard

The admin dashboard provides a platform overview with aggregated metrics such as active users, total orders, and total revenue. It supports monitoring and management from a single entry point.

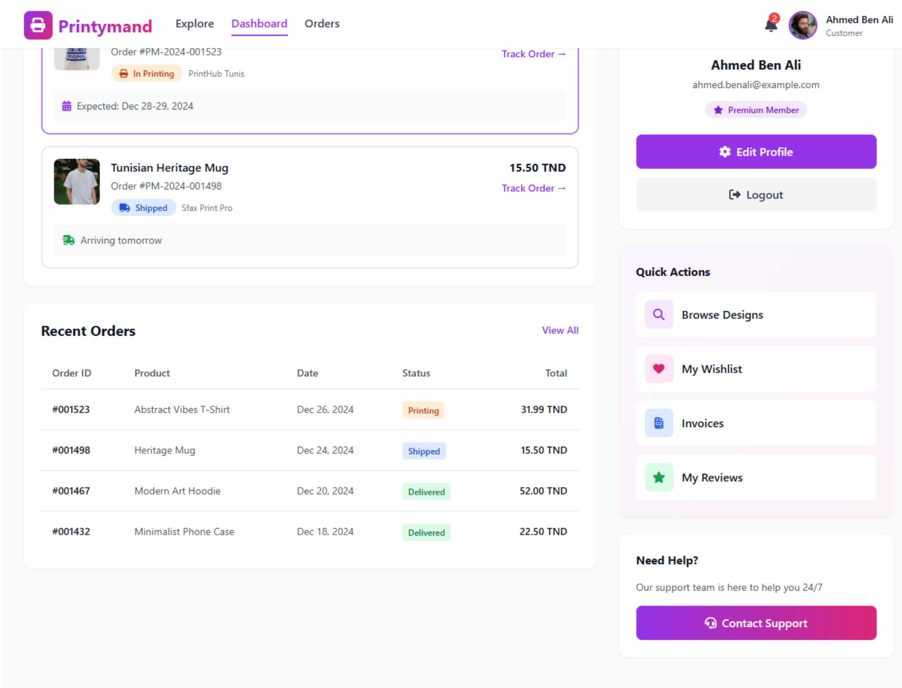

Figure 5.2: Cycle 3 : Customer dashboard interface 2

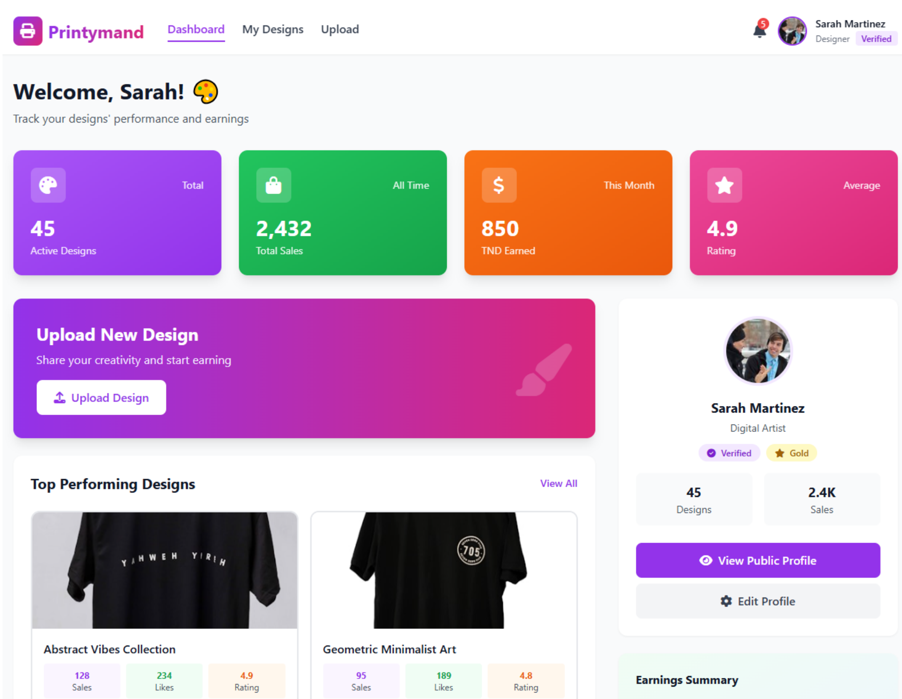

Figure 5.3: Cycle 3 : Designer dashboard interface

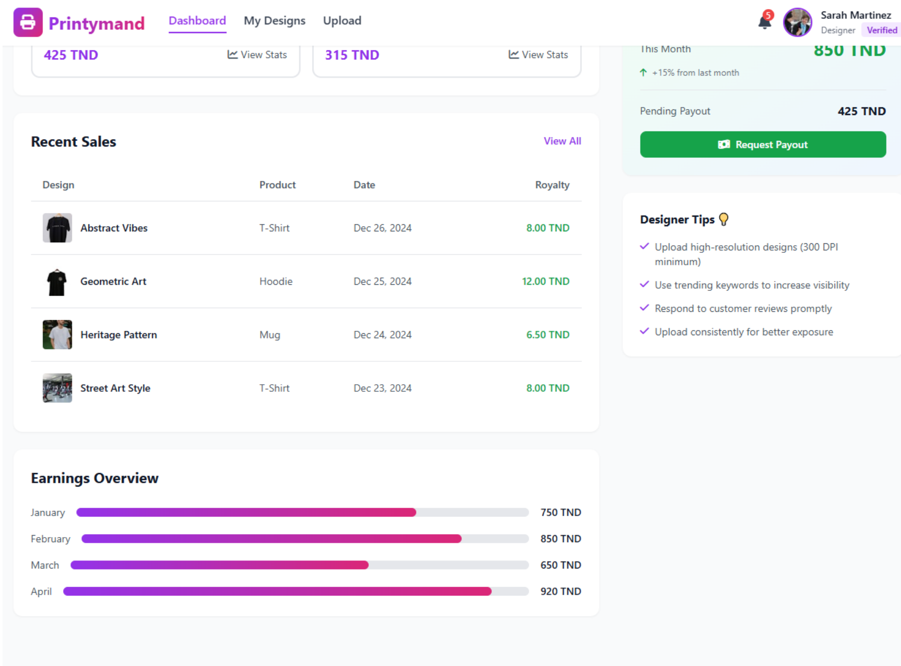

Figure 5.4: Cycle 3 : Designer dashboard interface 2

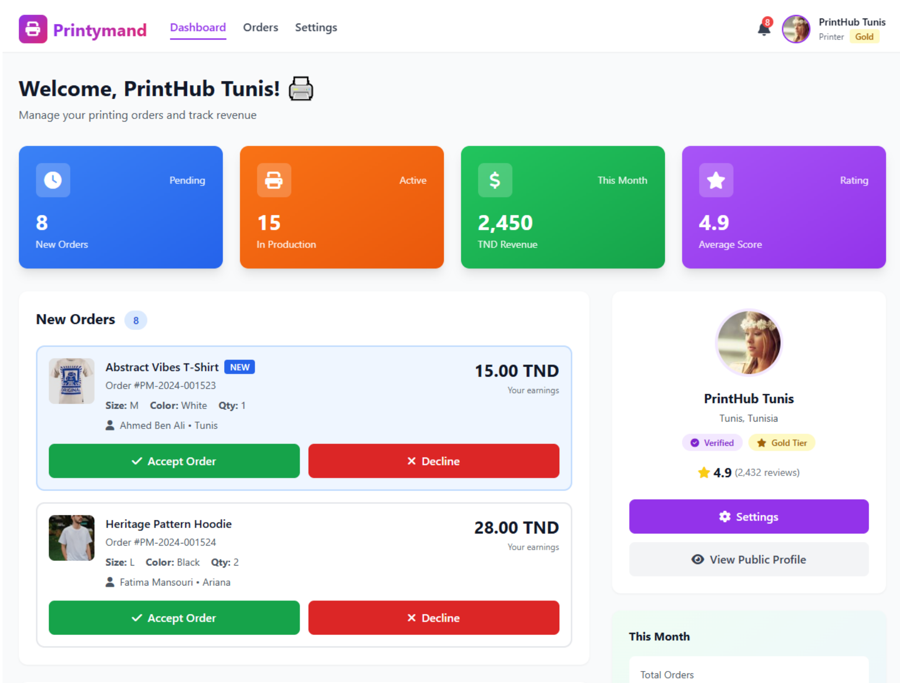

Figure 5.5: Cycle 3 : Printer dashboard interface

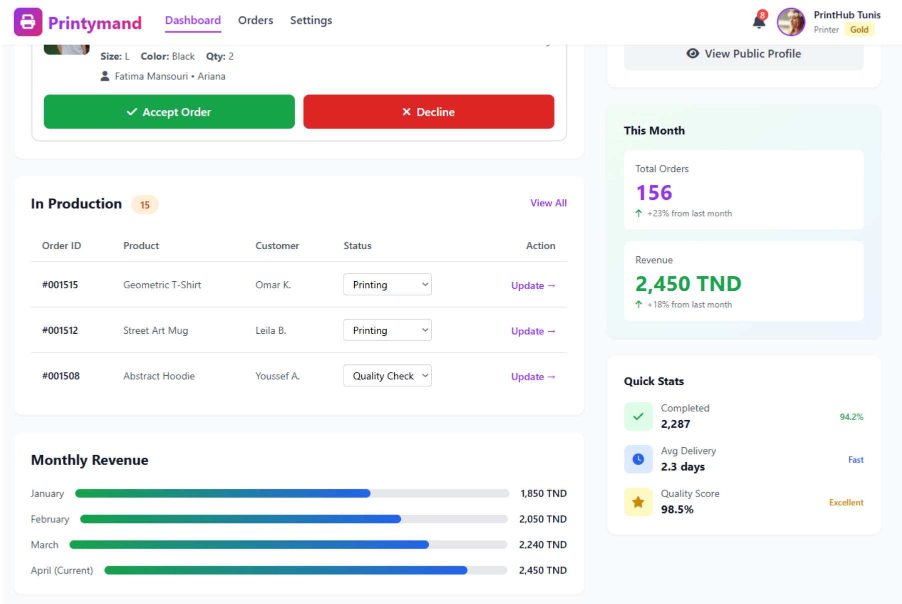

Figure 5.6: Cycle 3 : Printer dashboard interface 2

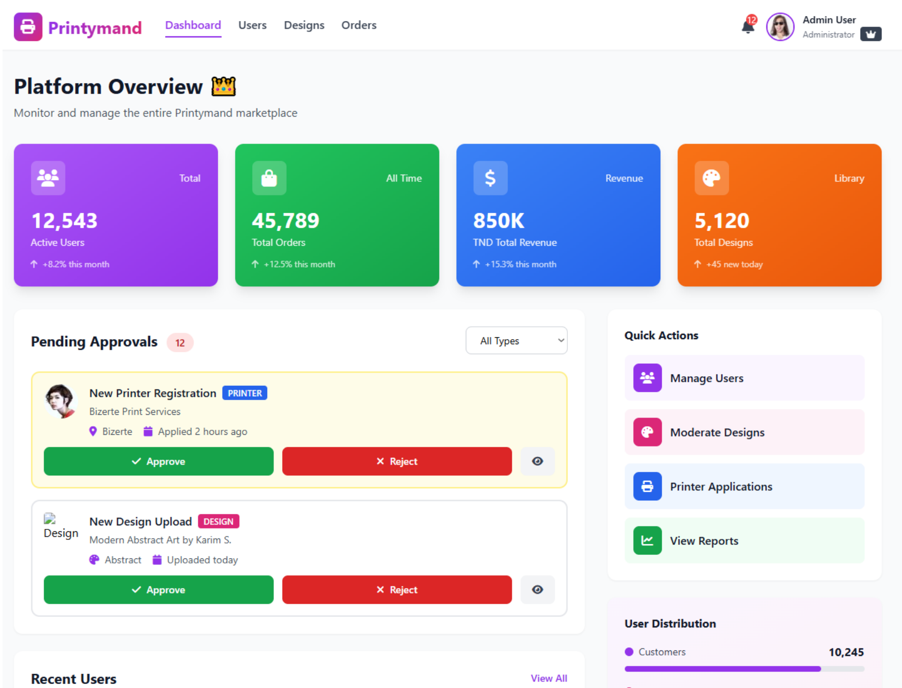

Figure 5.7: Cycle 3 : Admin Dashboard Interface

5.4 Modeling artifacts

This section describes the implemented capabilities of Cycle 3.

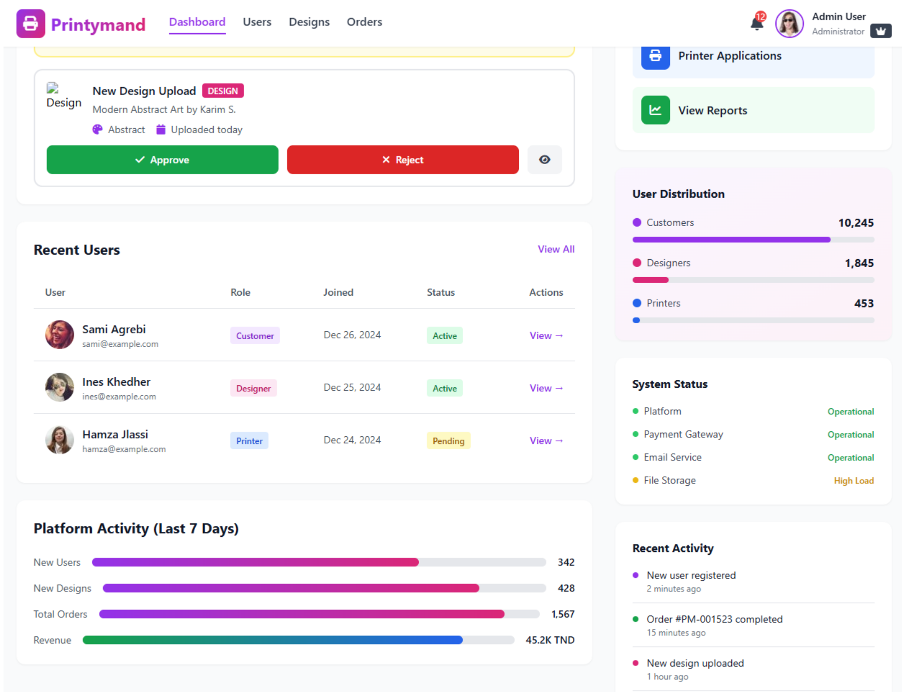

Figure 5.8: Cycle 3 : Admin dashboard interface 2

5.5 Validation and evaluation

In this section, we evaluate Cycle 3 using representative dashboard behaviors. The objective is to confirm that each role has a dedicated operational view with clear metrics and navigation.

5.5.1 Customer dashboard overview

The customer dashboard presents high-level indicators such as total orders, delivered orders, in-transit orders, and wishlist items. It provides a clear starting point to review active orders and quickly navigate back to the marketplace.

5.5.2 Designer dashboard overview

The designer dashboard emphasizes activity and revenue: active designs, total sales, total earnings (TND), and rating. This role-centered view supports creators by presenting performance indicators and an upload/manage entry point.

5.5.3 Printer dashboard overview

The printer dashboard focuses on operational fulfillment: it highlights new orders, orders in production, printer revenue, and average score. This structure supports the printer's daily workflow by keeping the most relevant operational metrics visible.

5.5.4 Admin dashboard overview

The admin dashboard provides a platform overview that aggregates metrics such as active users, total orders, and total revenue. This allows administrators to monitor platform health and identify areas requiring attention.

5.6 Conclusion

In conclusion, Cycle 3 consolidated *printymand* by delivering role-specific dashboards for Customer, Designer, Printer, and Admin. These views improve operational clarity, strengthen governance through administrative oversight, and provide a maintainable baseline for future platform evolution.

General Conclusion

The *printymand* integration project has been a valuable experience that strengthened our skills in full-stack web development, database design, and software engineering practices. Building a multi-role marketplace required careful analysis of requirements, consistent architectural decisions, and disciplined implementation across the frontend and backend. Tackling technical and organizational challenges reinforced the importance of iteration, testing, and clear communication.

The practical value of the platform is reflected in the workflow it supports: designers can publish designs and track royalties, customers can customize products and choose printers, and printers can manage fulfillment operations. By connecting these stakeholders through a single system, *printymand* demonstrates how a well-designed marketplace can enable transparency and scalability while reducing manual coordination between parties.

Throughout the project, we also gained insights into project management and teamwork. Effective collaboration, regular progress tracking, and feedback-driven refinement were essential to delivering meaningful outcomes within a limited timeframe.

In conclusion, this project has been a significant part of our academic journey. It demonstrated the impact that a carefully engineered platform can have on supporting real-world processes and stakeholders. The work provides a solid foundation for future improvements, such as deeper analytics, expanded product options, and further automation of marketplace governance.

Webography

[1] uml, *What is uml*, https://www.uml.org/what-is-uml.htm.
[2] V. Code, *Vs code documentation*, https://code.visualstudio.com/docs.
[3] Git, *Git official website*, https://git-scm.com.
[4] GitHub, *Github official website*, https://docs.github.com/en/get-started/start-your-journey/about-github-and-git.
[5] Wikipedia, *Adobe illustrator*, https://en.wikipedia.org/wiki/Adobe_Illustrator.
[6] Wikipedia, *Notion (productivity software)*, https://en.wikipedia.org/wiki/Notion_(productivity_software).
[7] Angular, *Angular documentation*, https://angular.dev.
[8] NestJS, *Nestjs documentation*, https://docs.nestjs.com.
[9] Wikipedia, *Html*, https://en.wikipedia.org/wiki/HTML.
[10] M. W. Docs, *Css: Cascading style sheets*, https://developer.mozilla.org/en-US/docs/Web/CSS.
[11] M. W. Docs, *Javascript*, https://developer.mozilla.org/en-US/docs/Web/JavaScript.
[12] T. Labs, *Tailwind css documentation*, https://tailwindcss.com/docs.
[13] P. G. D. Group, *Postgresql documentation*, https://www.postgresql.org/docs/.
[14] Prisma, *Prisma documentation*, https://www.prisma.io/docs.